

Product Requirements Document (PRD): Firma Core Platform MVP

Document purpose: This PRD defines the product scope, features, and requirements for the Firma platform—a client-centric document delivery and collaboration workspace for professional services. It is intended as a client and stakeholder deliverable for alignment on scope, behaviour, and acceptance criteria.

Audience: Product owners, stakeholders, and implementation teams.

Recent updates: Firm/Engagement terminology; canonical routes `/e/` (redirect from `/p/`); Search (Elasticsearch + semantic), Doc Comments, Bookmarks, Notifications (broadcast, dismiss), Reminders, Dashboard with action centre; connector abstraction for OneDrive; Sandbox/demo workspace; Board & Dossier tabs (BETA).

1. Overview

Firma is a professional client portal that connects organisations' existing Google Drive to a structured workspace. Users organise work by clients and projects, control access via roles and personas, and deliver documents through a branded experience—without migrating files or changing where data lives. This document describes the implemented and planned features for the Core Platform MVP and the billing model that supports tiered plans (Standard, Pro, Business, Enterprise).

2. Onboarding Flow

Purpose: Ensure every user has a dedicated workspace immediately upon signup.

- ✓ **Trigger:** New user signup via Auth provider (Google).
- ✓ **Routing:**
 - ✓ If User has *NO* organization memberships -> Redirect to `/onboarding`.
 - ✓ If User has existing memberships -> Redirect to `/d` (organizations list page).
- ✓ **Feature: "Name your Workspace":**
 - ✓ Simple form input for Organization Name.
 - ✓ Auto-generates a URL-friendly slug.
 - ✓ **Action:** Provisions a new Organization (Tier: Free) and assigns the user as `ORG_OWNER`.

3. Firm Level (formerly Organization)

Purpose: The root container for all business data, settings, and team access.

- ✓ **Structure:**
 - ✓ **Multi-Tenancy:** Users can belong to multiple firms (switching via User Profile dropdown; active firm in JWT as `active_firm_id`).
 - ✓ **Role-Based Access Control (RBAC):**
 - ✓ `firm_admin` : Firm-level administrative access (Billing, Settings, Client Creation). **No overarching engagement/document access** (must be an explicit engagement member).

- ☒ `firm_member` : Standard internal staff access.
- ☒ External/collaborator: Restricted access (Engagement-scoped).
- ☒ **Firms List (/d):**
 - ☒ Shows all firms the user belongs to in grid/list view.
 - ☒ Default firm highlighted.
 - ☒ "Create" for additional workspaces.
 - ☒ Clicking a firm navigates to `/d/o/[slug]` (clients list).
- ☒ **Dashboard:**
 - ☒ App Sidebar: Engagements (Projects), Members, Shares, Insights, Sources, Connectors.
- ☒ **Connectors:** Firm-level cloud storage at `/d/o/[slug]/connectors` . Connector registry supports multiple types (Google Drive implemented; OneDrive adapter stubbed). Used for engagement folder sync and Import from Drive. **RLS:** Connectors scoped by firm; storage adapters implement `IConnectorStorageAdapter` ; document permission regrant is provider-specific (OneDrive equivalent can be added later).
 - ☒ **Folder Setup:** Google Drive connector creates `.firma` root and firm folder with owner-only permissions. See [File Management Security](#).

4. Client Management

Purpose: Organise work by the customer or entity being served.

- ☒ **Data Model:** Clients are children of an Organization.
- ☒ **UI Components:**
 - ☒ **Sidebar Selector:** Dropdown to switch between active Clients.
 - ☒ **Persistence:** Remembers the last selected Client per Organization (LocalStorage).
 - ☒ **Empty State:** Prompts to create a Client if none exist.
- ☒ **Feature: Create Client:**
 - ☒ **Access:** `ORG_OWNER` only.
 - ☒ **Input:** Client Name, Industry/Sector.
 - ☒ **Output:** Creates a Client entity in DB.

5. Engagement Management (primary unit of work; data model still "Project")

Purpose: The primary unit of work (engagement, case, or audit). Canonical URLs use `/e/[engagementSlug]`; legacy `/p/` redirects to `/e/`.

- ☒ **Data Model:** Engagements (DB: `projects`) belong to a Client and are linked 1:1 with a connector root folder (e.g. Google Drive); the folder ID is stored in `connectorRootFolderId` .
- ☒ **UI Components:**
 - ☒ **Engagement List:** View all engagements for the active Client.
 - ☒ Toggle: Grid View / List View.
 - ☒ **Sidebar Navigation:**

- ☒ Expands to show active Engagement tabs (Files, Members, Shares, Comments, Insights, Sources, Audit, Settings) when inside an engagement.
- ☒ **Feature: Create Project:**
 - ☒ **Input:** Project Name, Start Date, Description.
 - ☒ **Integration:** Automatically creates a structured Google Drive Folder ([Client Name] / [Project Name]).
 - ☒ **Sync:** Stores the connector root folder ID in DB (connectorRootFolderId) for direct access.
- ☒ **Project Creator as Project Lead:** When a project is created, the creator is automatically added as a Project Lead member. This ensures the org owner (or any user who creates the project) sees the file list and has full project access without special-case logic; file visibility remains strictly persona-based.
- ☒ **Project Settings (Project Lead only):**
 - ☒ **Settings icon:** Prominent settings icon beside the project name (large bold title); visible only to users with the Project Lead persona for that project.
 - ☒ **Role access config:** In-code JSON config (PROJECT_ROLE_ACCESS.canViewProjectSettings: ['Project Lead']) determines who can view and use project settings; intended to move later to permissions or project_personas table.
 - ☒ **Settings modal:**
 - **Project Properties:** Name, Description, Save. When the project is closed (isClosed: true), property fields and Save are disabled; user must reopen to edit.
 - **Close project** (amber): Sets isClosed: true . Project becomes view-only for current members. All project members who are org guests (ORG_GUEST) are removed and their Google Drive folder access is revoked.
 - **Reopen project** (amber): Shown when isClosed: true . Sets isClosed: false ; only Project Lead can reopen.
 - **Delete project** (red): Soft delete. Sets isDeleted: true ; removes all project members; revokes all Google Drive permissions on the project folder (restrict to owner only). The project folder is **not** deleted in Google Drive so the org admin can still access files natively outside Firma. No other DB records or Drive files are deleted.
- ☒ **Project lifecycle flags:** Project.isClosed (Boolean, default false) and Project.isDeleted (Boolean, default false). Deleted projects are excluded from hierarchy and all project fetches; they are hidden from everyone in the portal.
- ☒ **Project Workspace (Tabs):** Tab and sidebar visibility are restricted by persona. See §7.6 **Permission-based UI: Who can see what** for the full matrix.
 1. ☒ **Files:** Document management (default view); full file browser, uploads, Import from Drive, folder upload. Visible to all personas with project access.
 2. ☒ **Members:** Member list, invitations, personas (see §7). Visible to Team Member, Project Lead, Client Owner, Org Owner only.
 3. ☒ **Shares:** Document sharing activity dashboard; per-document sharing workflows (accept/finalize/unlock). Same visibility as Members.
 4. ☒ **Insights:** Project-level insights dashboard (recent/trending/storage/sharing views). Same visibility as Members.

5. ☐ **Sources:** Data sources & connectors (placeholder; org-level Connectors at `/f/[slug]/connectors` for Google Drive). Same visibility as Members.
6. ☒ **Audit:** Project-level audit log shown in the Audit tab (main content). Visible to all personas with project access. Events are append-only and not editable (see §5 Audit view).
7. ☒ **Settings:** Project properties, close/reopen, delete. Visible to Project Lead, Client Owner, Org Owner only; implemented as a tab after Sources.
8. ☒ **Board** (BETA): Kanban view of engagement status, due dates, contract type, and tags. Requires `enableBetaFeatures` org flag; visible to internal personas only.
9. ☒ **Dossier** (BETA): Rich-text wiki pages per engagement. Requires `enableBetaFeatures` org flag; visible to internal personas only.

6. File Management

Purpose: Secure, robust, and familiar document handling powered by Google Drive.

- ☒ **Architecture:** "Headless" Drive integration. Firma acts as the UI, Google Drive acts as the storage/backend.
- ☒ **Security:**
 - ☒ **Direct-to-Drive Uploads:** Files are streamed directly from Browser to Google (TLS 1.3), bypassing Firma servers (Resumable Upload Protocol).
 - ☒ **Scoped Access:** System uses a Service Account or OAuth Scope limited to specific folders.
 - ☒ **Folder Permission Restrictions:**
 - ☒ **Centralized Access Control:** All folder permissions are managed exclusively through Firma Portal project membership. Google Drive's native sharing is disabled to prevent unauthorized access.
 - ☒ **.firma Root Folder:** When created during organization setup, permissions are restricted to owner-only with no inheritance from parent folder. All non-owner permissions are removed to ensure strict access control.
 - ☒ **Organization Folders:** Each organization folder created under `.firma` is restricted to owner-only access with no inheritance from `.firma` parent. This ensures complete isolation of organization data.
 - ☒ **Project-Level Permissions:** Access to project folders is granted automatically when users join projects via invitation (Project Lead and Team Member personas receive `can_edit` access). Permissions are revoked automatically when members are removed from projects.
 - ☒ **Enforcement:** Applied automatically during `setupOrgFolder()` when Google Drive connector is initialized. Both new and existing folders are secured to prevent unauthorized access. This ensures that all access is controlled through Firma Portal's project membership system, not through Google Drive's native sharing mechanisms.
- ☒ **Feature: File Browser:**
 - ☒ **Visuals:** Clean, table-based layout (Name, Owner, Date modified, File size). Column headers in Title case; Sort column right-aligned with row action menu.
 - ☒ **Icons:** Dynamic file-type icons (PDF, Sheets, Docs, Images, etc.).

- ☒ **Navigation:** Breadcrumbs for folder traversal.
- ☒ **Loading state:** Toolbar (Add, Type, People, Modified, Source, Search) disabled while the file list is loading.
- ☒ **Filters:** Type (multi-select with checkboxes; "All types" indeterminate when some selected; stays open until Done), People (Any / Owned by me / Not owned by me), Modified (Any / 7d / 30d / 1y), Source (filter by connector). All filter dropdowns use `text-xs` and header + Done button.
- ☒ **Sort:** Dedicated Sort column (icon + "Sort" label) with dropdown: Sort by (Name, Date modified, Date modified by me, Date opened by me), Sort direction (A–Z, Z–A), Folders (On top, Mixed with files). Default: Name, A–Z, Folders on top.
- ☒ **Refresh:** Button next to search to refresh list (e.g. after renaming in Google Docs).
- ☒ **Actions:** Row action menu (Preview, Edit in Google Docs, Open Folder in Drive, Download, Share, **Comments** (doc only), Version history, Bookmark, Set Due Date, Rename/Copy/Move/Delete when callbacks provided). Menu items `text-xs` ; menu stays open until user dismisses. **Comments** opens the right pane with the document-level DocComments thread (see §6 DocComments).
- ☒ **Long names:** Truncated file names show full name in tooltip.
- ☒ **Direct-to-Drive:** Add menu indicates uploads go directly to Google (never through Firma servers).
- ☒ **Feature: Add Menu:**
 - ☒ **New folder** (and New Doc/Sheet/Slide/etc. via submenu).
 - ☒ **From your computer:** Upload files; Upload folder (retains folder structure in Drive; in-app confirmation modal).
 - ☒ **Import from Google Drive:** Opens Google Picker with **My Drive** and **Shared Drives** tabs; starts at root (top-level folders/files), LIST view; user traverses hierarchy and multi-selects; selection flows to import confirmation dialog (copy/shortcut).
- ☒ **Feature: Upload Experience:**
 - ☒ **UX:** Bottom-right floating progress modal (similar to Gmail/Drive).
 - ☒ **Capabilities:**
 - ☒ **Multi-file Batching:** Queue multiple files.
 - ☒ **Background Processing:** Continue navigating while uploading.
 - ☒ **Conflict Resolution:** Detects duplicates. User can choose to **Rewrite** (Replace) or **Keep Both** (Auto-rename).
 - ☒ **Smart Tracking:** If a file is renamed (e.g. `Report.pdf` -> `Report_1.pdf`) due to conflict, the UI captures the *final* name.
 - ☒ **Post-Upload:** "Show file location" button highlights the new file in the list.
- ☐ **Feature: File Assignment to External Members** (High Priority):
 - ☐ **Purpose:** Enable Project Leads to assign individual files/folders from the `general` folder to External Collaborator and Client Contact personas. This provides granular file-level sharing beyond project-level access.
 - ☐ **Access Control:** Only Project Leads can assign files to external members. External Collaborator and Client Contact personas do not automatically receive Drive access upon project invitation; they only get access to files explicitly assigned to them.

- ☐ **Assignment UI:**
 - ☐ When Project Lead selects a file/folder in the file browser, show "Assign to Members" option in the row action menu.
 - ☐ Assignment modal lists External Collaborator and Client Contact members for the project.
 - ☐ Allow multi-select assignment (assign one file to multiple external members).
 - ☐ Show visual indicators (badge/icon) in file browser for files that have been assigned to external members.
- ☐ **Google Drive Permission Grant:**
 - ☐ When a file is assigned to an External Collaborator, automatically grant `writer` (`can_edit`) permission to that specific file/folder in Google Drive.
 - ☐ When a file is assigned to a Client Contact, automatically grant `reader` (`can_view`) permission to that specific file/folder in Google Drive.
 - ☐ Permissions are scoped to the assigned file/folder only, not the entire project folder.
- ☐ **Assignment Tracking:**
 - ☐ Track file assignments in database (new `FileAssignment` table or extend existing models) to record which files are assigned to which external members.
 - ☐ Store: `fileId` (Google Drive file ID), `projectId`, `memberId` (ProjectMember ID), `assignedAt`, `assignedBy` (userId).
- ☐ **Revoke Assignment:**
 - ☐ Allow Project Lead to revoke file assignments, which removes Google Drive permissions for that file/folder.
 - ☐ Revocation removes the assignment record and revokes the Drive permission.
- ☐ **Scope Limitations:**
 - ☐ File assignment only applies to files/folders within the `general` folder (not `confidential` folder).
 - ☐ Cannot assign files to Project Lead or Team Member personas (they already have full access).
 - ☐ Assignment is file-level, not folder-level inheritance (assigning a folder does not automatically assign all files within it).
- ☒ **Feature: Document-level DocComments:**
 - ☒ **Purpose:** Async, append-only comment thread per document for feedback or conversation. Not real-time; stored in DB and visible to all personas with project access.
 - ☒ **Entry:** "Comments" in the document row action menu (documents only, not folders). Opens the right pane with the DocComments thread.
 - ☒ **Behaviour:** Messages are **not editable** (enforced at API and DB: no UPDATE on message records). Users can add new comments only. Author is derived from session (authorUserId stored; email/persona resolved by join at read time).
 - ☒ **Data:** Stored in `DocCommentMessage` (`organizationId`, `clientId`, `projectId`, `projectDocumentId`, `authorUserId`, `content`, `createdAt`). No `updatedAt`; DB trigger prevents UPDATE.
- ☒ **Feature: Project-level Audit view:**

- ☒ **Purpose:** Immutable, append-only log of project-scoped events (e.g. file uploaded, status change, shared externally, project closed). Rendered in the right pane.
 - ☒ **Entry:** "Audit" tab in the project workspace. The tab content shows the project audit log.
 - ☒ **Behaviour:** Audit records are **not editable** (enforced at API and DB: no UPDATE on audit records). Event types include project lifecycle (created, updated, closed, reopened, soft-deleted), membership changes, document added/removed, document activity status change, document shared externally/finalized/regranted, indexing, and config (connector folder attached, settings changed). Actor is stored as actorUserId; email/name derived by join at read time.
 - ☒ **Data:** Stored in `PlatformAuditEvent` (scope PROJECT for this MVP; organizationId, clientId, projectId, projectDocumentId optional, eventType, eventAt, actorUserId, metadata). No updatedAt; DB trigger prevents UPDATE.
- ☐ **Feature: Document Review & Collaboration** (High Priority):
 - ☐ **Review System:** Add comments/feedback functionality with threaded discussions to maintain conversation context.
 - ☐ **Approve/Finalize/Publish Workflow:** Allow guests (External Collaborator, Client Contact) to approve, finalize, or publish documents.
 - ☐ **Publish/Finalize to Lock & Version:** When a document is published/finalized, lock it and create a version snapshot.
 - ☐ **Export to PDF:** Enable export of finalized documents to PDF format.
 - ☐ **Watermark Branding:** Add watermarking with organization branding to exported PDFs.
 - ☐ **"Track" Tab:** Add a "Track" tab beside files to show review status, comments, approvals, and version history.
- ☐ **Feature: Project Templates & Duplication** (High Priority):
 - ☐ **Template Projects:** Define template projects with pre-defined folder structures and template documents. Provide ready-made templates for targeted Lines of Business (LOBs).
 - ☐ **Template Selection:** Allow users to choose a template project when starting a new project to begin with reusable assets.
 - ☐ **Duplicate Project:** Enable duplication of existing projects with all folder structures, documents, and configurations.
- ☐ **Feature: Document Relationships & Dependencies** (High Priority):
 - ☐ **Related/Dependent Documents:** Support commitment-based or linked access between documents, not just folder-level access.
 - ☐ **Relationship Management:** Add relationship tracking amongst folders or files (e.g., parent-child, dependencies, references).
 - ☐ **Relationship Tree View:** Display relationship tree visualization showing document dependencies and connections (project task-like structure).
- ☐ **Feature: Client Communication & Follow-ups** (High Priority):
 - ☐ **Automated Consolidated Follow-ups:** Send automated consolidated client follow-up emails on all pending documents.

- ☐ **Custom Follow-up Messages:** Allow customization of follow-up message templates and scheduling.
- ☐ **Calendar Integration:** Block calendar through Calendly (or similar) for document discussion scheduling.
- ☐ **Bi-directional Calendar Requests:** Enable both Team → Client and Client → Team calendar request flows for document discussions.
- ☐ **Feature: UI Enhancements** (High Priority):
 - ☐ **Project Card Images:** Add random/featured images to Project cards for visual appeal and better project identification.

6a. Search

Purpose: Let users find documents and folders across projects quickly.

- ☒ **Project-level Search:** Full-text Elasticsearch search within a project's Drive folder via in-app search panel.
- ☒ **Semantic Search:** AI-powered semantic search using `lib/search-utils.ts` and `semantic-search.ts`.
- ☒ **Batch Indexing:** Files are indexed on upload/import via Inngest background jobs (`file.index.requested` , `file.index.batch.requested`).
- ☒ **Scan & Index:** Recursive project scan indexes entire folder trees on project creation or connector setup (`project.index.scan.requested`).

6b. Bookmarks

Purpose: Allow users to save and quickly return to frequently accessed files and engagements.

- ☒ **User Bookmarks** (`/d/u/bookmarks`): Persistent per-user bookmarks with a maximum of 50 entries.
- ☒ **Bookmark API:** `GET/POST/DELETE /api/bookmarks` for full bookmark management.
- ☒ **File Bookmarking:** "Bookmark" action available in the file browser row action menu.

6c. Notifications & Reminders

Purpose: Keep team members informed of important events without leaving the portal.

- ☒ **Notifications Page** (`/d/u/notifications`): System notifications with priorities (INFO, WARNING, CRITICAL); unread count badge.
- ☒ **Broadcast Notifications:** Org Admins can broadcast scoped notifications (org / client / project level) from the notification centre.
- ☒ **Realtime Delivery:** Supabase Realtime subscription pushes new notifications to connected clients instantly.
- ☒ **Reminders Page** (`/d/u/reminders`): User-level reminders with metadata tracking and status management.

6d. Dashboard & Action Centre

Purpose: Provide a unified landing page with pending actions, recent activity, and business insights.

- ☒ **Dashboard** (`/dash`): Landing page after login showing action centre (pending shares, approvals, due dates), business insights panels, and file review modals.
- ☒ **Recent Items** (`/d/u/recent`): Tracks and displays recently visited clients and engagements (stored in `localStorage`).
- ☒ **Action Centre**: Surfaces documents awaiting review or approval across all active engagements.

6e. Beta Workspace Features

Purpose: Advanced engagement views gated behind `enableBetaFeatures` org flag.

- ☒ **Board Tab** (BETA): Kanban/board view of engagement status, kickoff/due dates, contract type, and tags. Visible only to users with `project:can_view_internal` capability.
- ☒ **Dossier/Wiki Tab** (BETA): Rich-text wiki pages per engagement (`EngagementWikiPage` model). Supports persistent page count; visible to internal personas only.
- ☒ **Sandbox / Demo Workspace**: Automatically provisioned demo firm ("Acme Corp" sample data) for new free-tier users to explore the product without creating real data. Populated via Inngest background jobs with sample Drive files.

7. RBAC & Permission System

Purpose: Implement comprehensive Role-Based Access Control (RBAC) with hierarchical permissions and in-memory caching for optimal performance.

7.1 Code-Based RBAC Architecture

- ☒ **Simplified Persona Mapping**: Moved from a complex DB-heavy schema to a code-based single source of truth (`persona-map.ts`).
- ☒ **Consolidated Personas**:
 - ☒ **Internal**: `org_owner` , `org_member` , `sys_admin` .
 - ☒ **Project**: `project_admin` (Project Lead), `project_editor` (Team Member), `project_viewer` (Guest/Viewer).
 - ☒ **External**: `proj_ext_collaborator` , `proj_guest` (Sharing-scoped).
- ☒ **Capability-Based Permissions**: Personas map to specific technical capabilities:
 - `org:can_manage` : Full org settings control.
 - `client:can_manage` : Full client management control.
 - `project:can_manage` : Project settings and member management.
 - `project:can_view_internal` : Access to internal tabs (Members, Shares, Insights).
 - `project:can_view` : Base project access (Files tab).

7.2 Permission Computation & Performance

- ☒ **Autonomous Project Management**: Org Owners no longer have "override" access to projects. They must be explicitly joined to a project (as `project_admin`) to see documents or internal tabs.
- ☒ **In-Memory Caching**: User permissions are cached in `UserSettingsPlus` on login for optimal performance (~0-5ms lookup).
- ☒ **Zero DB Queries on Navigation**: All route gates use the cached capability set.
- ☒ **Cache Invalidation**: Automatic invalidation on any membership or setting change.

7.3 Permission Retrieval Flow

1. **Login:** User signs in → `buildUserSettingsPlus()` server action called → Single DB query fetches all permissions → Cached in-memory
2. **Navigation:** All routes check permissions from cache (no DB queries)
3. **UI Gating:** Components receive permissions as props, gate actions based on `canEdit`, `canManage`, `canView`
4. **Permission Changes:** Cache automatically invalidated, rebuilt on next request

7.4 UI Feature Flagging

- ☒ **Upload/Create Actions:** Gated with `canEdit` permission
- ☒ **Settings Button:** Gated with `canManage` permission
- ☒ **Member Actions:** Invite, remove, change role gated with `canManage`
- ☒ **Client Creation:** Gated with `client` scope `can_manage` permission
- ☒ **Organization Selector:** Shows all organizations user belongs to, defaults to `isDefault` organization

7.5 Deployment Version Management

- ☒ **Session Invalidation on Code Changes:** System tracks deployment versions to invalidate sessions when code changes
- ☒ **Version Sources:**
 - `DEPLOYMENT_VERSION` environment variable (CI/CD)
 - `NEXT_PUBLIC_BUILD_TIMESTAMP` (build-time)
 - `DEV_SERVER_START_VERSION` (development-time)
- ☒ **Middleware Validation:** Checks deployment version cookie on each request
- ☒ **Cache Rebuild:** On version mismatch, session cleared and user redirected to login for fresh permission cache

7.6 Permission-based UI: Who can see what

Project-level UI (tabs and sidebar sub-menus) is restricted by persona. The following matrix defines **who can see what** for the project workspace. This is the product source of truth for permission-based UI.

Project tabs and sidebar sub-menus (by persona/capability):

UI element	Guest (<code>project_viewer</code>)	Ext. Collab (<code>proj_ext_collaborator</code>)	Editor (<code>project_editor</code>)	Lead (<code>project_admin</code>)	Org Owner (<code>org_owner</code>)
Files	✓	✓	✓	✓	(M)
Members	✗	✗	✓	✓	(M)
Shares	✗	✗	✓	✓	(M)
Insights	✗	✗	✓	✓	(M)
Sources	✗	✗	✓	✓	(M)
Settings	✗	✗	✗	✓	(M)

*(M) = Membership Required. Org Owners ONLY see these if they are explicit project members.

- **Files:** Anyone with `project:can_view` capability.

- **Members, Shares, Insights, Sources:** Internal collaborative tabs; requires `project:can_view_internal`.
- **Settings:** Administrative; requires `project:can_manage`.

View As: Org Owners can use "View as" (sidebar dropdown) to simulate another persona for testing; the UI and API respect the selected persona (via cookie) so that tab and sidebar visibility match the matrix above.

Extensibility: New tabs or personas are added via a configuration-driven permission framework (see HLD). The matrix above is implemented as capability rules (e.g. `project:can_view`, `project:can_view_internal`, `project:can_manage`) resolved from RBAC.

8. Project Members & Personas

Purpose: Manage access control through granular, role-based personas at the project level.

- ☒ **File visibility:** Only users who are project members with a persona (Project Lead, Team Member, etc.) see project files. There is no special path for "org member with no project persona"; the project creator is always added as a Project Lead so they see files from creation.
- ☒ **Data Model (New):**
 - ☒ **ProjectPersona** : Helper table scoped to `Organization`. Defines role templates.
 - **Fields:** `name`, `description`, `role` (System Role: `ORG_MEMBER` | `ORG_GUEST`), `permissions` (JSON: `can_view`, `can_edit`, `can_manage`, `can_comment`), `isDefault`.
 - ☒ **ProjectMember Update:** Links `userId` to `projectId` AND `personaId`. Added `settings` (JSONB).
 - ☒ **ProjectInvitation** : Tracks pending invites (`email`, `personaId`, `status`, `token`).
- ☒ **Default Personas (Consolidated):**
 - Project Lead (`project_admin`)**
 - **Permissions:** `project:can_view`, `project:can_view_internal`, `project:can_manage`.
 - **Description:** Internal lead responsible for project oversight and member management.
 - Team Member (`project_editor`)**
 - **Permissions:** `project:can_view`, `project:can_view_internal`.
 - **Description:** Internal staff with full workspace access but no admin rights.
 - External Collaborator (`proj_ext_collaborator`)**
 - **Permissions:** `project:can_view`.
 - **Description:** Restricted external partner with access only to shared files.
 - Guest/Viewer (`project_viewer`)**
 - **Permissions:** `project:can_view`.
 - **Description:** Client stakeholder with read-only/comment access.
- ☒ **UI Components (Members Tab):**
 - ☒ **Member List:** Shows User, Persona, and Status.
 - ☒ **Invitation Modal:** Input Email + Select Persona.

- ☒ **Change Member Role:** Dialog to update an existing member's persona assignment.
- ☐ **Persona Renaming** (Low Priority):
 - Allow Project Leads to rename default persona names per project (e.g., rename "Team Member" to "Accountant" or "External Collaborator" to "Contractor").
 - Simple rename field in Project > Members screen, edit button next to persona name.
 - **Scope Limitation:** Rename only - no new persona creation, no permission changes, no description editing.
- ☒ **Invitation Status:**
 - **PENDING** : Show "Resend" button.
 - **ACCEPTED** : Link Verified / Signup in progress.
 - **JOINED** : Fully onboarded member.
- ☒ **Invitation Flow:**
 - ☒ **Sender:** Invites email -> Selects Persona -> System sends email (SMTP/Brevo).
 - ☒ **Invitee:** Clicks link -> If new, registers -> If existing, signs in -> Redirected to Project.
 - ☒ **System:** Updates `ProjectInvitation` status -> Creates `ProjectMember` record -> Assigns `OrganizationMember` role if needed.
 - ☒ **Google Drive Access:** Automatically grants folder permissions based on persona:
 - ☒ **Project Lead & Team Member:** Receive `writer` (`can_edit`) access to project's Google Drive folder upon joining.
 - ☒ **External Collaborator & Client Contact:** No automatic Drive access (view-only through Portal UI). Files must be explicitly assigned to these personas via File Assignment feature (see [File Management](#)).
 - ☒ **Removal:** When any member is removed from a project, their Google Drive folder access is automatically revoked. File assignments are also revoked when a member is removed.
 - ☒ **Security (Tamper-Proofing):**
 - **Backend Enforcement:** Invitation redemption logic explicitly validates that the `invite.email` matches the `authenticated_user.email`.
 - **Prevention:** Prevents "Link Forwarding" or "UI Tampering" where a user attempts to claim an invite intended for another email address.
 - ☒ **Reliability (Transactional Integrity):**
 - **Atomic Operations:** Acceptance logic wrapped in `prisma.$transaction`.
 - **Guarantee:** Ensures Member Creation, Org Member Creation, and Invite Status Update happen as a single atomic unit. Prevents inconsistent states (e.g. "Joined" status without "Member" record).

9. Non-Functional Requirements: Error Handling & Logging

Purpose: Ensure system reliability, observability, and robust error recovery across all application layers.

- ☒ **Infrastructure:**
 - ☒ **Sentry Integration:**
 - ☒ **Client-Side:** Browser error tracking with Session Replay (10% sampling).

- ☒ **Server-Side:** Node.js runtime error tracking with performance monitoring.
- ☒ **Edge Runtime:** Middleware error tracking.
- ☒ **Source Maps:** Automated upload during build for readable stack traces.
- ☒ **Logger Utility:**
 - ☒ **Centralized Logging:** Unified interface (`lib/logger.ts`) for all log levels (`debug` , `info` , `warn` , `error`).
 - ☒ **Environment Awareness:**
 - **Dev:** Colored console output with metadata.
 - **Prod:** JSON structured logs + Sentry integration.
 - ☒ **Context Awareness:** Captures user ID, organization context, and breadcrumbs.
- ☒ **Resilience & Recovery:**
 - ☒ **Global Error Boundaries:**
 - ☒ **Root Level:** Catches top-level crashes (`app/global-error.tsx`).
 - ☒ **Page Level:** Catches route-specific errors (`app/error.tsx`).
 - ☒ **Component Level:** Granular boundaries for widget isolation (e.g., `ProjectFileList`).
 - ☒ **Graceful Degradation:** Failed components display user-friendly fallback UI without crashing the entire app.
- ☒ **Privacy & Security:**
 - ☒ **Sanitization:**
 - ☒ Sentry masking for text input and sensitive data.
 - ☒ Removal of PII from production logs.
 - ☒ **Health Checks:** Filtered out from error tracking to reduce noise.
 - ☒ **Database Row-Level Security (RLS):**
 - **Multi-tenancy:** At organization level (tenant = Organization). RLS applied at different levels for different tables.
 - **Org-level RLS:** For org-owned tables (`organizations` , `clients` , `projects` , `connectors` , `project_personas` , `organization_members`) restrict by `organization_id` using helper functions. **Connectors:** `portal.connectors` has organization-level RLS (policy `connectors_org_isolation`).
 - **Project-level RLS:** For project-scoped tables (`project_members` , `project_invitations`) restrict by project membership. See [HLD § Security & Compliance – RLS multi-tenancy strategy](#).
 - **Implementation:** RLS policies enforce hierarchical isolation: Organization → Client → Project → Document. Helper functions (`get_current_user_organization_ids()` , `is_user_project_member()` , etc.) enable efficient policy evaluation.

10. Waitlist System

Purpose: Build anticipation for the Pro plan launch, collect early interest, and drive viral growth through referrals.

10.1 Waitlist Signup Flow

- ☒ **Public Waitlist Page** (/waitlist):
 - ☒ **Fixed Email Field:** Always visible at top of page with "Enter your email" label.
 - ☒ **Email Validation:** Real-time validation checks email format and waitlist status (debounced 800ms).
 - ☒ **Email Locking:** After validation completes, email field locks with lock icon and pencil edit button.
 - ☒ **Dynamic Content Branching:**
 - ☒ **Branch 1 - Existing Waitlist Member:** Shows waitlist status (position, points, referrals), referral link, and leaderboard.
 - ☒ **Branch 2 - New Signup:** Shows registration form (company name, size, role, comments) with email pre-filled (hidden field).
 - ☒ **Branch 3 - Success:** Shows confirmation message after successful submission.
 - ☒ **Social Proof Counter:** Displays "X people have already joined!" with avatars when total count > 25.
 - ☒ **Early Access Offer Banner:** Prominently displays Pro Plus upgrade benefits and referral rewards.
- ☒ **Form Fields:**
 - ☒ **Email** (required): Pre-filled from email check, hidden in form.
 - ☒ **Company Name** (optional): Text input.
 - ☒ **Company Size** (optional): Dropdown (Solo, 2-10, 11-50, 51-200, 200+).
 - ☒ **Role** (optional): Text input (e.g., Consultant, Accountant, Founder).
 - ☒ **Comments** (optional): Textarea for additional context.
 - ☒ **Honeypot Field:** Hidden field to prevent bot submissions.
 - ☒ **Cloudflare Turnstile:** Captcha verification for spam prevention.
- ☒ **Security & Validation:**
 - ☒ **Rate Limiting:** 3 submissions per hour per IP address.
 - ☒ **Duplicate Prevention:** Checks for existing email before submission.
 - ☒ **Server-Side Validation:** Email format, Turnstile verification, honeypot check.
 - ☒ **Privacy:** Email addresses masked in logs (first 3 chars + ***).

10.2 Referral System

- ☒ **Referral Code Generation:**
 - ☒ **Format:** 8-character alphanumeric code (uppercase, excludes confusing chars: 0/O, 1/l).
 - ☒ **Uniqueness:** Database constraint ensures unique codes.
 - ☒ **Auto-Generation:** Created automatically on waitlist signup.
- ☒ **Referral Benefits Structure:**

For Referrers (People who share their link):

- ☒ **Position Boost:** Move up 3 positions in waitlist per successful referral.
- ☒ **Points System:** Earn 30 points per referral (used for leaderboard ranking).
- ☒ **Pro Plus Upgrade:** Automatically upgraded to Pro Plus at 5 referrals.
- ☒ **Discount:** 20% off first 3 months when subscribing.

For Referees (People who sign up via referral link):

- ☒ **Skip Ahead:** Start 10 positions higher than normal signups (via createdAt adjustment).
- ☒ **Priority Access:** Early access when Pro plan launches.
- ☒ **Discount:** 15% off first 3 months when subscribing.
- ☒ **Referral Link Sharing:**
 - ☒ **Format:** `https://firma.one/waitlist?ref=ABC123XY`
 - ☒ **Display:** Prominently shown in waitlist status view.
 - ☒ **Copy Functionality:** One-click copy button with visual feedback.
 - ☒ **URL Parameter Handling:** Automatically detects and processes referral codes from URL.
- ☒ **Position Calculation:**
 - ☒ **Normal Signup:** Position = count of people who signed up before them (based on createdAt).
 - ☒ **Referral Signup:** Position = count before them - 10 positions (via createdAt timestamp adjustment).
 - ☒ **Referrer Boost:** For each referral, referrer's positionBoost increments by 3 (tracked in database).

10.3 Leaderboard & Social Proof

- ☒ **Leaderboard Display:**
 - ☒ **Top 10 Referrers:** Ranked by referral count (points = referrals × 30).
 - ☒ **Table Format:** POSITION | EMAIL | POINTS columns.
 - ☒ **Rank Badges:** Special styling for #1 (gold), #2 (silver), #3 (bronze).
 - ☒ **User Highlighting:** Current user's row highlighted with purple background.
 - ☒ **User Inclusion:** If user not in top 10, their row shown below with "..." separator.
 - ☒ **Privacy:** Email addresses masked (first 3 chars + ***).
- ☒ **User Status Display:**
 - ☒ **Current Position:** Shows waitlist position number.
 - ☒ **Current Points:** Calculated as referrals × 30.
 - ☒ **Referral Count:** Number of successful referrals.
 - ☒ **Position Boost:** Total positions gained from referrals.
 - ☒ **Points to Move Up:** Shows how many more points needed to advance (if applicable).
- ☒ **Social Proof Counter:**

- ☒ **Display Trigger:** Shows when total waitlist count > 25.
- ☒ **Visual Elements:** 3 most recent joiners' avatars (initials) + "+X more" badge.
- ☒ **Count Display:** Large, bold number with "people have already joined" text.

10.4 Database Schema

- ☒ **Waitlist Table** (`admin.waitlist`):
 - ☒ **Core Fields:** `id` , `email` , `plan` (default: "Pro"), `createdAt` .
 - ☒ **Profile Fields:** `companyName` , `companySize` , `role` , `comments` (all optional).
 - ☒ **Referral Fields:**
 - ☒ `referralCode` (String, unique, 8 chars): Unique code for sharing.
 - ☒ `referredBy` (String, nullable): `referralCode` of person who referred them.
 - ☒ `referralCount` (Int, default: 0): Number of successful referrals.
 - ☒ `positionBoost` (Int, default: 0): Total positions gained from referrals.
 - ☒ **Security Fields:** `ipAddress` (for rate limiting).
 - ☒ **Indexes:** `email` , `plan` , `createdAt` , `referralCode` , `referredBy` .

10.5 User Flows

- ☒ **New User Signup Flow:**
 1. User visits `/waitlist` .
 2. Enters email in fixed field at top.
 3. System validates email format and checks waitlist status (debounced).
 4. Email locks after validation.
 5. If not found: Form appears with email pre-filled (hidden).
 6. User completes profile (company, role, etc.).
 7. Completes Turnstile captcha.
 8. Submits form.
 9. Success message displayed.
 10. Email remains locked.
- ☒ **Existing User Status Flow:**
 1. User enters email in fixed field.
 2. System checks waitlist status.
 3. Email locks after validation.
 4. Status view displays:
 - Position, points, referrals.
 - Referral link with copy button.
 - Leaderboard with user's position highlighted.
 - Points needed to move up (if applicable).
- ☒ **Referral Flow:**
 1. Existing waitlist member shares referral link (`/waitlist?ref=ABC123XY`).
 2. New user clicks link.
 3. Referral code detected in URL.
 4. Banner shows "You were referred!" message.
 5. User signs up via form.

6. System records `referredBy` field.
7. Referee skips ahead 10 positions.
8. Referrer's `referralCount` increments.
9. Referrer's `positionBoost` increases by 3.
10. Both users see updated positions.

- ☒ **Email Edit Flow:**

1. User clicks pencil icon on locked email field.
2. Email unlocks for editing.
3. Status cleared (allows re-check).
4. User edits email.
5. System re-validates and checks status.
6. Email locks again after validation.

10.6 Server Actions

- ☒ **`submitWaitlistForm()`** : Handles form submission with rate limiting, honeypot, Turnstile verification, duplicate checks, and referral processing.
- ☒ **`getWaitlistStatus()`** : Returns user's waitlist status (position, ahead/behind counts, referral stats).
- ☒ **`getWaitlistLeaderboard()`** : Returns top 10 referrers ranked by points, user's rank, and total count.
- ☒ **`getWaitlistCount()`** : Returns total waitlist count and recent joiners for social proof.

10.7 Admin Features

- ☒ **Internal Waitlist View** (`/internal/waitlist`):
 - ☒ **Statistics Dashboard**: Total count, breakdown by plan (Pro, Pro Plus, Business, Enterprise).
 - ☒ **Waitlist Table**: Shows all entries with email, plan, company info, referral stats, and signup date.
 - ☒ **Sorting**: Sortable by date, plan, referral count.
 - ☒ **Access**: Internal admin dashboard only.

10.8 Future Enhancements

- ☐ **Email Notifications**: Notify users when someone uses their referral link.
- ☐ **Referral Analytics**: Track referral click-through rates and conversion metrics.
- ☐ **Social Sharing**: Direct share buttons for Twitter, LinkedIn, Email.
- ☐ **Referral Milestones**: Badges and achievements for referral milestones (5, 10, 25 referrals).
- ☐ **A/B Testing**: Test different benefit structures and messaging.
- ☐ **Waitlist Management**: Admin tools to manually adjust positions, send bulk emails, export data.

11. Billing, subscriptions & payment (Polar)

Purpose: Enable paid plans (Standard, Pro, Business, Enterprise) with one provider, keep subscription state in sync, and support upgrade, renewal, and cancellation.

Payment gateway: **Polar** (polar.sh) — hosted checkout and customer portal.

Where details live (no duplication in this PRD):

- Tiers, features, roadmap: [prd-subscriptions.md#plans-and-feature-distribution](#) and `frontend/config/pricing.ts`.
- Checkout, webhooks, DB fields, acceptance criteria: [prd-subscriptions.md#polar-integration-and-data-contract](#).
- Architecture map: [hld-subscription.md](#).

Data model (summary): Subscription **state** (status, plan name, pricing model, period end, Polar customer/subscription ids) is stored on **platform.subscriptions**, keyed by **billing anchor** `firmId`. **platform.firms** holds **billing caps and grouping** (e.g. engagement limits, `billingSharesSubscriptionFromFirmId`) — not a second copy of Polar subscription columns. UI and session hints read the active subscription row (and caps) without calling Polar on every request.

Webhooks: `POST /api/webhooks/polar` verifies Polar signatures and updates **platform.subscriptions** (and related hooks). Idempotent handling for retries.

Outcome: Customers manage billing in Polar; the app reflects the correct plan and limits after events sync.